

통합반 9회 · 아이나루 관제 화면과 배포

- [이번 회차에서 할 일](#)
- [1. 시작 전 준비 \(복습·환경 확인\)](#)
- [2. 핵심 개념](#)
- [3. 활동 1 — 제공 화면의 설비·이벤트 데이터 연결 · 50분](#)
- [4. 활동 2 — 선택 사건의 업무 시작·상태 조회 버튼 연결 · 50분](#)
- [5. 활동 3 — 준비된 서버에 게시하고 앱 URL 확인 · 50분](#)
- [6. 활동 4 — 시뮬레이터 조치·재조회와 재시작 연습 · 50분](#)
- [7. 실습 과제](#)
- [8. 완료 확인 체크리스트](#)
- [9. 회고 \(10분\)](#)
- [10. 실제 플랫폼 대응](#)
- [강사 노트](#)
- [용어](#)

날짜·시간	2027-02-03 (수) 09:00~13:00	모듈	M4 · 원 일정 20번
블록	확장 B9 / 플랫폼 B9	산출물	플랫폼에서 여는 교육용 앱
완료 확인	앱 URL에서 같은 사건의 데이터와 업무 진행을 조회		

회차별 블록 로드맵 — 하나의 시스템에 매 회차 기능을 더한다

구현 새로 만든다 · **확장** 조건·연결을 바꾼다 · **플랫폼** 교육용 플랫폼에 연결한다

실전반 4학년 · 9회 × 4시간 (합수·질의·정책 수정과 검증)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 10/31	구현	구현	구현						
2회 11/07		확장	확장						
3회 11/21			확장	구현					
4회 11/28		확장	확장		구현				구현
5회 01/25				확장		구현	구현		
6회 01/26			플랫폼	확장+P			확장		
7회 01/27						확장+P	확장+P	구현+P	
8회 01/28	확장			확장			확장		
9회 01/29									확장+P

통합반 2·3학년 · 10회 × 4시간 (제공 코드의 매핑·설정·조건 완성)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 11/14	구현	구현							
2회 01/25		확장	구현						
3회 01/26				구현					
4회 01/27		확장	확장		구현				구현
5회 01/28				확장		구현			
6회 01/29							구현	구현	
7회 02/01			플랫폼	확장+P					
8회 02/02						확장+P	확장+P	확장+P	
9회 02/03									확장+P
10회 02/04	확장			확장			확장	확장	

이번 회차가 다루는 블록

이번 회차에서 할 일

4회에 만든 관제 화면(B9)은 로컬 관제 API 한 곳만 읽었다. 이번 회차의 제조 운영 앱은 세 곳을 함께 읽는다. 설비·사건 목록은 7회에 발행한 Ontology Studio 객체에서, 사건 상세와 온도 추이는 관제 API 에서, 업무 진행은 8회에 연결한 프로세스에서 가져온다. 화면 코드(Vue)는 제공본을 그대로 쓰고, 우리는 **어디서 읽을지 정하는 설정과 주주의 빈칸만** 채운 뒤 교육용 플랫폼(Lab Platform)의 Apps 기능으로 게시한다. 끝나면 앱 URL 하나로 같은 사건의 데이터·업무 진행·조치 기록을 확인할 수 있다.

학습 목표

- 앱 설정 `window.APP_CONFIG` 의 네 값이 각각 어느 서버·스키마·프로세스를 가리키는지 설명할 수 있다.
- 화면의 설비 목록·사건 목록·사건 상세·업무 인스턴스 조회 주소를 채우고 실제 GET 결과로 확인할 수 있다.
- 앱을 게시하고 게시 로그의 네 단계(검증·번들·상태 확인·게시 완료)를 읽을 수 있다.
- 시뮬레이터 조치 후 앱에서 다시 조회해 화면·프로세스·조치 기록이 같은 `event_id` 를 가리키는지 확인할 수 있다.

1. 시작 전 준비 (복습·환경 확인)

```
cd lecture/system
curl -s http://localhost:8910/studio/schemas | python3 -c "import sys,json;s=json.load(sys.stdin)[0];print(s['schema_name'],s['published_version'])"
curl -s http://localhost:8910/process/definitions | python3 -c "import sys,json;print([d['def_id'] for d in json.load(sys.stdin)])"
```

예상 출력 (2026-09-14 실행):

```
HydraulicOps 1
['cooling_response']
```

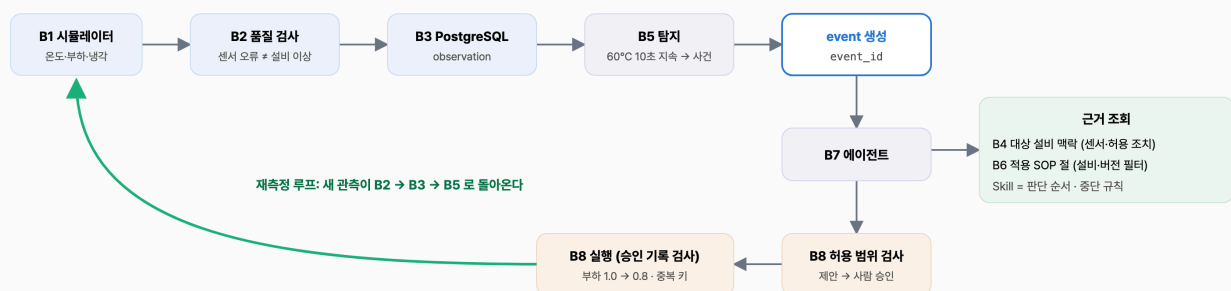
스키마가 발행돼 있지 않으면(`None`) 앱의 설비 목록이 비고, 7회에 본 409 거부가 난다. 먼저 강사에게 알린다.

복습 질문: 8회 인스턴스 기록에서 "업무 결과: 종결"과 "설비 재측정: RECOVERED" 는 왜 따로 표시됐나?

2. 핵심 개념

하나의 사건이 통과하는 경로

종결 기준은 도구 호출 성공이 아니라 조치 이후 새 관측의 회복이다. 모든 기록은 같은 `event_id` 로 이어진다.



B9 관제 같은 `event_id` 로 감지 · 근거(도구 호출·SOP 인용) · 승인 · 명령 결과 · 재측정 판정 을 한 화면에 모은다 — 회복이면 종결, 미개선이면 이관, 센서가 끊기면 판정 보류

하나의 사건이 통과하는 경로

제조 운영 앱은 `system/labplatform/app_templates/ops-console/` 의 정적 파일 세 개(`index.html` , `vue.global.prod.js` , 게시 때 생기는 `config.js`)로 되어 있다. 화면이 읽는 곳을 표로 정리하면 다음과 같다.

화면 칸	읽는 곳
설비 (Ontology: Asset)	POST {platform_api}/studio/schemas/{schema_name}/objects/Asset/fetch (읽기 전용 조회)
사건 (Asset — HAS_EVENT→ Event)	GET {platform_api}/studio/schemas/{schema_name}/objects/Asset/{asset_id}/related/HAS_EVENT
온도 추이	GET {hydops_api}/api/series?asset_id=...&seconds=180
사건 기록	GET {hydops_api}/api/events/{event_id}
업무 진행	GET {platform_api}/process/instances/{instance_id}, GET {platform_api}/process/worklist
업무 열기 버튼	POST {platform_api}/process/definitions/{process_def_id}/start

화면은 2초마다(`setInterval(this.tick, 2000)`) 온도 추이·사건 목록·선택 사건을 다시 읽는다.

주의 — 앱 게시는 화면만 올리는 일이다. 분석·시뮬레이터 서버(:8800)와 교육용 플랫폼(:8910)은 강사가 미리 운영한다. 게시가 성공해도 바인딩한 서버가 멈춰 있으면 화면은 비어 보인다. 게시 로그의 "상태 확인" 단계가 그 차이를 알려 준다.

주의 — "같은 사건 확인" 세 개의 ✓는 설비 회복을 뜻하지 않는다. 화면 event_id·프로세스 입력 event_id·조치 기록 event_id 가 같다는 연결 검사일 뿐이다. 회복 여부는 사건 기록의 "재측정" 줄로 읽는다.

3. 활동 1 — 제공 화면의 설비·이벤트 데이터 연결 · 50분

목표

`labs/integrated/S09/config.js` 와 `app_bindings.js` 의 빈칸을 채우고, 설비 목록과 사건 목록 주소를 실제로 조회한다.

따라 하기

1. 제공 화면의 조회 코드를 읽는다(`ops-console/index.html` 에서 발췌).

```
const cfg = window.APP_CONFIG;
const S = `${cfg.platform_api}/studio/schemas/${cfg.schema_name}`;
// ...
async loadAssets() { this.assets = (await post(`${S}/objects/Asset/fetch`, { limit: 10 })).objects; },
async loadEvents() { const r = await fetch(`${S}/objects/Asset/${this.assetId}/related/HAS_EVENT`).then(r => r.json()); ... },
```

2. `config.js` 네 칸을 채운다: 관제 API 주소, 교육용 플랫폼 주소, 7회 스키마 이름, 8회 프로세스 정의 ID.

3. `app_bindings.js` 의 `assetList · assetEvents` 빈칸(클래스 이름, 관계 이름)을 채운다.
4. 데이터 담당이 채운 주소를 직접 조회한다.

```
curl -s http://localhost:8910/studio/schemas/HydraulicOps/objects/Asset/HYD-01/related/HAS_EVENT |  
python3 -c "import sys,json;d=json.load(sys.stdin);print(d['count']);  
[print(o['status'],o['event_type'],o['event_id']) for o in d['objects']]"
```

5. 검사한다: `PYTHONPATH= . .venv/bin/python -m pytest ../labs/integrated/S09 -q -k "blanks or config or bindings"`

확인 포인트

사건 목록 조회 결과(2026-09-14):

```
6  
CLOSED COOLING_ANOMALY EVT-20260914-131041-HYD-01-f81e  
REJECTED COOLING_ANOMALY EVT-20260914-130541-HYD-01-61ac  
ESCALATED COOLING_ANOMALY EVT-20260914-130444-HYD-01-64c4  
CLOSED COOLING_ANOMALY EVT-20260914-130416-HYD-01-07b0  
SENSOR_CHECK SENSOR_FAULT EVT-20260914-130352-HYD-01-2edd  
HOLD_NO_EVIDENCE PRODUCT_QUALITY EVT-20260914-134527-HYD-01-6294
```

설비 목록(Asset fetch)은 `count 3` — HYD-01 유압설비 1호기, HYD-02 유압설비 2호기, HYD-03 유압설비 3호기(신규)였
다. S09 테스트는 채운 주소를 제공 index.html 의 조회 주소와 **글자 단위**로 대조한다.

흔한 실수

- `hydops_api` 와 `platform_api` 를 바꿔 넣는 것. 사건 상세(`/api/events`)는 :8800, 객체·프로세스는 :8910 이다.
- 주소 끝에 `/` 를 붙이는 것. 화면이 `${cfg.hydops_api}/api/events/...` 로 이어 붙이므로 `//api/...` 가 된다.
- 사건 목록이 6건인데 설비 이상 사건이 6건이라고 읽는 것. 제품 품질 사건(PRODUCT_QUALITY)과 센서 오류 사건이 섞여
있다. 목록의 `event_type` 을 함께 읽는다.

4. 활동 2 — 선택 사건의 업무 시작·상태 조회 버튼 연결 · 50분

목표

사건을 고르면 사건 상세 → 대응된 업무 인스턴스 → 위크리스트 순서로 읽고, "업무 열기" 버튼이 같은 사건 값으로 업무를 시작하도
록 주소를 채운다.

따라 하기

1. 제공 화면의 두 메시지를 읽는다.

```
async refreshEvent() {  
  if (!this.eventId) return;  
  this.detail = await get(`${cfg.hydops_api}/api/events/${this.eventId}`);  
  const ref = this.detail.event.process_ref;  
  if (ref) this.instance = await get(`${cfg.platform_api}/process/instances/${ref.instance_id}`);  
  ...  
}
```

```

},
async openWork() {
  const e = this.detail.event;
  const r = await post(`${cfg.platform_api}/process/definitions/${cfg.process_def_id}/start`, { values: {
    event_id: e.event_id, asset_id: e.asset_id, run_id: e.run_id, event_type: e.event_type } });
  this.startMsg = r.duplicate ? `이미 시작된 업무 ${r.instance_id} 를 엽니다 (중복 시작 없음)` : `업무
    ${r.instance_id} 시작`;
  ...
},

```

2. `app_bindings.js` 의 `eventDetail · processInstance · openWork` 빈칸을 채운다.
3. 에이전트·조치 담당은 `same_event.py` 의 빈칸을 채운다. 화면의 computed `sameUi · sameProcess · sameAction` 과 같은 판정이다.
4. 사건 상세와 인스턴스를 GET 으로 읽는다.

```

curl -s http://localhost:8800/api/events/EVT-20260914-131041-HYD-01-f81e | python3 -c "import
  sys,json;d=json.load(sys.stdin);print(d['event']['process_ref']);print([a['event_id'] for a in
  d['actions']])"
curl -s http://localhost:8910/process/instances/PI-75788b6a | python3 -c "import
  sys,json;i=json.load(sys.stdin);print(i['status'],i['business_result'],i['equipment_outcome'],i['vars']
  ['event_id'])"

```

5. 테스트를 모두 돌린다: `PYTHONPATH=. .venv/bin/python -m pytest ../labs/integrated/S09 -q`

확인 포인트

2026-09-14 실행 결과:

```

{'key': 'cooling_response:EVT-20260914-131041-HYD-01-f81e:1', 'def_id': 'cooling_response',
'instance_id': 'PI-75788b6a'}
['EVT-20260914-131041-HYD-01-f81e']
COMPLETED 종결 RECOVERED EVT-20260914-131041-HYD-01-f81e

```

확인	읽은 값	결과
화면 event_id	<code>detail.event.event_id</code> = ...-f81e	✓
프로세스 입력 event_id	<code>instance.vars.event_id</code> = ...-f81e	✓
조치 기록 event_id	<code>actions[0].event_id</code> = ...-f81e	✓

반대로 로컬 모드에서 처리된 사건(예: ...-07b0)은 `process_ref` 가 `null` 이다. 이때 화면은 인스턴스를 읽지 않고 "업무 열기" 버튼을 보여 준다.

흔한 실수

- "업무 열기"를 여러 번 누르면 인스턴스가 여러 개 생긴다고 생각하는 것. 인스턴스 키 `cooling_response:event_id:1` 이 이미 있으면 새로 만들지 않고 기존 인스턴스를 연다(화면 문구 "중복 시작 없음").
- 이미 끝난 로컬 사건(`process_ref` 가 `null` 인 CLOSED-ESCALATED 사건)에서 "업무 열기"를 누르는 것. 버튼은 보이지만 새 업무는 **열린 사건**에만 시작한다. 수업에서는 새로 감지된 사건에서만 누른다.
- 조치 기록이 없는 사건(거부·센서 오류)에서 세 번째 확인이 ✕ 라고 적는 것. 화면은 "— 조치 기록 없음 (실행 전 종료)"로 표시하고, `same_event.py` 는 `None` (해당 없음)을 돌려준다.

5. 활동 3 — 준비된 서버에 게시하고 앱 URL 확인 · 50분

목표

Apps 화면에서 앱을 게시하고, 게시 로그와 앱 URL 에서 같은 설정이 쓰였는지 확인한다.

따라 하기

1. 콘솔 **Apps** 에서 이름 `hydops-ops` , 템플릿 `ops-console` 을 확인하고 바인딩 줄
(`hydops_api=http://localhost:8800` · `platform_api=http://localhost:8910` ·
`schema=HydraulicOps` · `process=cooling_response`)이 우리 `config.js` 와 같은지 본다.
2. 감사 신호에 맞춰 **게시** 를 한 번 누른다.
3. 게시 로그와 배포된 설정을 GET 으로 읽는다.

```
curl -s http://localhost:8910/apps/hydops-ops/logs | python3 -c "import sys,json;[print(x['at']  
[11:19],x['message'],x['detail']) for x in reversed(json.load(sys.stdin))]"  
curl -s http://localhost:8910/apps/hydops-ops/config.js
```

4. 브라우저에서 앱 URL `http://localhost:8910/apps/hydops-ops/` 을 열고 HYD-01 → 사건 ...-f81e 를 누른다.

The screenshot shows the 'Lab Platform' interface. On the left is a dark sidebar with a menu: '개요', 'DS Data Fabric', 'ON Ontology Studio', 'AG Agents', 'OR Processes', and 'OR Apps' (highlighted). The main area is titled 'Apps' with a subtitle '대용: 아이나루 앱 게시'. It features a form to '앱 게시' (Publish App) with input fields for 'hydops-ops' and a dropdown for 'ops-console', followed by a '게시' (Publish) button. Below the form, the binding string is displayed: '바인딩: hydops_api=http://localhost:8800 · platform_api=http://localhost:8910 · schema=HydraulicOps · process=cooling_response'. A section titled 'hydops-ops' shows a 'PUBLISHED' status and the URL 'http://localhost:8910/apps/hydops-ops/'. Below this, a log of events is shown with timestamps and messages: '13:10:04 검증: 템플릿과 바인딩 설정 확인', '13:10:04 번들: 정적 파일 복사와 config.js 생성', and '13:10:04 상태 확인: 바인딩된 서버 응답'. At the bottom left of the sidebar, there is a note: '일체 플랫폼을 수입용으로 제한된 환경입니다. 대용 기능은 각 화면 상단에 표시됩니다.'

앱 게시 로그

확인 포인트

게시 로그(2026-09-14, 오래된 순):

```

13:10:04 검증: 템플릿과 바인딩 설정 확인 {'bindings': ['hydops_api', 'platform_api', 'schema_name',
'process_def_id'], 'template': 'ops-console'}
13:10:04 번들: 정적 파일 복사와 config.js 생성 {'files': ['config.js', 'index.html', 'vue.global.prod.js']}
13:10:04 상태 확인: 바인딩된 서버 응답 {'hydops_api': 200, 'platform_api': 200}
13:10:04 게시 완료: /apps/hydops-ops/ {'status': 'PUBLISHED'}

```

배포된 `config.js`:

```

window.APP_CONFIG = {
  "hydops_api": "http://localhost:8800",
  "platform_api": "http://localhost:8910",
  "schema_name": "HydraulicOps",
  "process_def_id": "cooling_response",
  "app_name": "hydops-ops"
};

```



게시된 제조 운영 앱 — 같은 사건 확인 ✓✓✓

S18 화면에서 사건 기록 칸은 "상태 CLOSED · 제안 PROPOSE REDUCE_LOAD 0.8 · 근거 SOP-COOL-001 v2 §4, SOP-LOAD-001 v1 §4 · 승인 APPROVED kim.operator 0.8 · 명령 SUCCEEDED · 재측정 RECOVERED 55°C 이하 연속 56초", 업무 진행 칸은 "PI-75788b6a · COMPLETED · 업무 결과 종결 · 설비 RECOVERED" 를 보여 준다.

흔한 실수

- 바인딩 하나를 빼고 게시하는 것. 게시 코드(`labplatform/apps.py`)는 네 값 중 하나라도 없으면 422 "바인딩 누락"으로 멈춘다.
- 상태 확인이 200 이 아닌데 게시가 끝났다고 보고하는 것. 이때 상태는 `PUBLISHED` 가 아니라 `PUBLISHED_WITH_WARNINGS` 다.

- 앱 이름에 한글·밑줄을 쓰는 것. 영문·숫자·하이픈만 받는다.
- 다른 컴퓨터에서 앱을 열고 `localhost` 가 같은 서버라고 생각하는 것. 브라우저가 부르는 `localhost` 는 앱을 연 컴퓨터 자신이다.

6. 활동 4 — 시뮬레이터 조치·재조회와 재시작 연습 · 50분

목표

새 사건이 생기고 조치·재측정이 끝날 때까지 앱에서 다시 조회해 기록이 채워지는 순서를 보고, 서버 재시작 뒤에도 같은 사건을 같은 앱 URL 에서 다시 찾는다.

따라 하기

1. 강사가 관제 화면(:8800)에서 HYD-01 에 **냉각 성능 저하** 를 주입한다. 앱의 온도 추이가 60°C 점선 위로 올라가는지 본다.
2. 새 사건이 사건 목록에 나타나면 누르고, **업무 열기** 를 누른다(강사가 Watch 를 이미 돌렸다면 "이미 시작된 업무 ... 를 엽니다" 문구가 뜬다).
3. 업무 진행 칸에 "조치 승인" 카드가 뜨면 결정·승인값·승인자를 넣고 **제출** 한다.
4. 사건 기록 칸이 명령 → 재측정 순으로 채워지는 동안 기록 표에 시각을 적는다.

시각	사건 status	업무 현재 단계	명령 결과	재측정
(기록)	PENDING_APPROVAL	조치 승인	—	—
(기록)	VERIFYING	조치 후 재점검	SUCCEEDED	—
(기록)	CLOSED 또는 ESCALATED	종결 또는 담당자 이관	SUCCEEDED	RECOVERED 또는 NOT_IMPROVED

5. **재시작 연습(강사 시연)**. 강사가 `scripts/servers.sh platform 4` 로 두 서버를 다시 띄운다. 학생은 같은 앱 URL 을 새로고침하고, 방금 사건을 다시 눌러 기록을 대조한다.

확인 포인트

- 조치 후 재측정 기준은 교육용 가정값이다: 조치 30초 뒤부터 60초 창에서 55°C 이하가 10초 연속이면 RECOVERED(`hydops/config.py`). 8회 기록의 ...-f81e 는 `longest_below_s 56` , `valid_ratio 1.0` 으로 RECOVERED 였다.
- 명령 결과 줄(SUCCEEDED)이 먼저 생기고, 재측정 줄은 창이 끝난 뒤에 생긴다. 두 줄이 동시에 채워지지 않는다는 점을 표의 시각으로 확인한다.
- 재시작 뒤 코드상 달라지는 것과 남는 것은 다음과 같다. 수업에서는 실제로 본 결과를 표에 적는다.

항목	재시작 뒤	근거 코드
<code>run_id</code>	새 값	<code>Plant.reset</code> → 새 <code>PlantRuntime</code>
이전 실행의 열린 사건	<code>RUN_ENDED</code> 로 닫힘 (회복으로 기록하지 않음)	<code>store.end_open_events_of_other_runs</code>
온도 추이 그래프	새 실행 관측만 표시	<code>/api/series</code> 가 현재 <code>run_id</code> 로 조회

항목	재시작 뒤	근거 코드
사건 목록·사건 기록	이전 사건도 그대로 조회	HAS_EVENT 는 <code>run_id</code> 조건 없이 JOIN
게시 앱·프로세스 인스턴스·인스턴스 키	그대로	교육용 플랫폼 메타데이터가 PostgreSQL <code>labplatform.doc</code> 에 저장

흔한 실수

- 명령이 SUCCEEDED 로 바뀌자마자 "회복"이라고 발표하는 것. 심각한 저하(냉각 0.25)에서는 명령이 성공해도 재측정이 NOT_IMPROVED 로 끝나 이관된다(`out/e2e_platform_report.json` "조치 후 미개선").
- 재시작 뒤 온도 그래프가 비었다고 데이터가 지워졌다고 생각하는 것. 그래프는 현재 실행만 그린다. 이전 사건 기록은 사건 목록에서 그대로 열린다.
- 실패한 인스턴스가 남긴 열린 사건 때문에 새 사건이 안 생기는 경우를 서버 고장으로 보는 것. 설비·유형당 열린 사건은 하나라서 새 감지가 막힌다. 시뮬레이터 초기화가 이전 실행의 열린 사건을 `RUN_ENDED` 로 닫는 이유다.

7. 실습 과제

- 폴더: `labs/integrated/S09/` (`TASK.md` 참고)
- 빈칸 위치
 - `config.js` : `hydops_api` , `platform_api` , `schema_name` , `process_def_id`
 - `app_bindings.js` : 설비 목록 클래스(`Asset`), 사건 관계(`HAS_EVENT`), 사건 상세 서버 변수, 인스턴스 ID 변수, 업무 시작 메서드, 정의 ID 변수
 - `same_event.py` : `event_id` 필드 3곳, 인스턴스 변수 묶음 이름, 조치 기록 없음 표시, `process_ref` 필드
- 검증 방법: 게시 검증이 요구하는 네 값, 제공 `index.html` 과 주소 대조(정적), 게시된 앱의 `config.js` ·로그와 대조(GET), 채운 주소로 사건 상세·인스턴스를 읽어 세 가지 `event_id` 가 같은지 확인(GET). 테스트는 게시·업무 시작을 하지 않는다.
- 확인 명령: `cd lecture/system && PYTHONPATH=. .venv/bin/python -m pytest ../labs/integrated/S09 -q` (정답본 6 passed)
- 경험자 과제: 설비 HYD-02 를 기본 선택으로 바꾸려면 `index.html` 의 어느 값(`assetId: 'HYD-01'`)이 관여하는지 찾아 적는다(파일은 고치지 않는다).

8. 완료 확인 체크리스트

- ☐ `config.js` 네 값과 게시된 앱의 `config.js` 가 같다.
- ☐ 게시 로그 네 단계와 `PUBLISHED` 상태를 기록했다.
- ☐ 앱 URL 에서 HYD-01 사건 목록을 열었다.
- ☐ 한 사건에서 화면·프로세스 입력·조치 기록 `event_id` 세 개가 같음을 확인했다.
- ☐ 같은 사건의 업무 진행(인스턴스 ID·현재 단계·업무 결과)을 앱에서 읽었다.
- ☐ 명령 결과와 재측정 결과가 생긴 시각을 따로 기록했다.

9. 회고 (10분)

1. 앱이 세 곳(Ontology 객체·관계 API·프로세스)을 읽는 대신 한 곳만 읽는다면 무엇을 확인할 수 없게 되나?

2. 재시작 뒤에도 같은 사건을 다시 찾을 수 있었던 것은 어떤 키 덕분인가?
3. 게시가 "성공"인데 화면이 빈 경우, 어떤 순서로 원인을 좁혀 가겠나?

10. 실제 플랫폼 대응

교육용 플랫폼 기능	실제 플랫폼 기능	다른 점
<code>POST /apps {name, template, config} →</code> 검증·번들·상태 확인·게시 로그	아이나루 앱 생성·게시	교육용은 정적 파일 복사와 <code>config.js</code> 생성만 한다. 분석·시뮬레이터 서버는 강사가 사전 운영하고 학생은 화면만 게시한다
앱의 Ontology 객체 조회(Asset fetch, HAS_EVENT)	Ontology Studio 발행 객체를 앱에서 조회	교육용은 앱이 REST 주소를 직접 부른다
앱의 업무 열기·인스턴스·워크리스트 조회	Process GPT 업무 시작·진행 조회	교육용은 인증 없이 CORS 를 모두 허용한다
<code>GET /apps/{name}/logs</code>	아이나루 배포 로그	교육용 로그는 네 단계 메시지만 남긴다

강사 노트

시간 배분

시각	내용
09:00~09:50	활동 1 — 설정·설비/사건 주소 빈칸, GET 확인
09:50~10:00	휴식
10:00~10:50	활동 2 — 사건 상세·인스턴스·업무 열기 주소, <code>same_event.py</code> (역할 교대: 데이터 담당 ↔ 에이전트·조치 담당)
10:50~11:00	휴식
11:00~11:50	활동 3 — 게시, 로그, 앱 URL
11:50~12:00	휴식
12:00~12:50	활동 4 — 주입·승인·재조회, 재시작 시연
12:50~13:00	회고

사전 준비

- `scripts/servers.sh platform 4` 후 `scripts/bootstrap_platform.py --upto watch` (앱 게시는 학생이 한다). 리허설로 이미 게시했다면 같은 이름으로 다시 게시하면 폴더를 지우고 새로 만든다.
- 활동 2 확인용으로 프로세스로 처리된 사건 1건이 필요하다. 수업 전에 냉각 이상 1건을 주입해 Watch **한 번 실행** → 승인까지 끝내 둔다.
- 재시작 시연은 강사만 한다. 재시작하면 진행 중인 사건이 `RUN_ENDED` 로 닫히므로 활동 4 의 사건이 끝난 뒤에 한다.

장애 대응 게시가 실패하면 로컬 관제 화면(<http://localhost:8800>)으로 활동 4 의 주입·재조회를 계속한다. 앱 URL 조회는 플랫폼 실습 완료로 기록하지 않고 복구 후 보충한다.

용어

용어	뜻
제조 운영 앱	제공 Vue 화면 <code>ops-console</code> 을 게시한 교육용 앱. URL <code>http://localhost:8910/apps/hydops-ops/</code>
APP_CONFIG	게시 때 생성되는 <code>config.js</code> 의 설정 객체. 서버 주소·스키마·프로세스 정의를 담는다
바인딩	화면이 읽을 서버·스키마·프로세스를 정하는 설정 값
번들	게시할 정적 파일 묶음(<code>config.js</code> , <code>index.html</code> , <code>vue.global.prod.js</code>)
process_ref	사건에 기록된 업무 인스턴스 대응(<code>instance_id</code> , <code>key</code>)
같은 사건 확인	화면·프로세스 입력·조치 기록의 <code>event_id</code> 가 같은지 보는 세 개의 표시
RUN_ENDED	시뮬레이터 초기화로 닫힌 이전 실행의 열린 사건. 회복이 아니다